

# AURIX's Developer Role Guide



## Role : Web Frontend Developer

Assigned to: BHUMIKA

### 1. The "Why" of Your Component

You are building the "Face" and the "Door" of AURIX. Security tools often fail because they output massive, unreadable text walls that overwhelm developers. Your job is to make the highly complex AI reasoning, authentication, and exploit data visually intuitive, clean, and accessible from any browser. You are the only part of the system the end-user will actually see and interact with.

### 2. Your Domain in the Architecture (HLD & LLD References)

As the Web Frontend Developer, your work is defined strictly by the client-side architectural boundaries:

- **HLD Ownership:** You own the **Web Dashboard** inside **Layer 1 (Client Interfaces)** of the [aurix-HLD.mmd](#) or [png](#) diagram. You communicate *only* with Layer 2 (Bhavya's API Server).
- **LLD Ownership (API Flows):** You are responsible for implementing the frontend side of the [webDashboard-api-sequence-diagram.mmd](#) or [png](#). This means you must build the HTTP logic to POST `/api/scans/github` and set up the polling loop for GET `/api/scans/{scan_id}`.
- **LLD Ownership (Database UI Mapping):** While you do not build the database, you must map your UI directly to the VERIFIED\_VULNERABILITIES table from the updated ER diagram. Specifically, you must render the AI's logic\_reasoning, display the wargame\_status, show the poc\_script and patch\_code, and provide a UI toggle to update the is\_resolved boolean.
- **LLD Ownership (Authentication - "The Door"):** You own the "Door" to the system. Bhavya (Role 2) manages the secure "Vault" in the database, but *you* must build the visual Login and Sign-Up pages, capture the user's credentials, send them to Bhavya's API, and securely store the returned session token (JWT) in the browser to keep the user logged in.

### 3. Step-by-Step Guide

1. **Setup & Scaffolding:** Initialize a Next.js or React application with Tailwind CSS for rapid styling.
2. **Build the "Door" (Auth UI):** Create the `<LoginForm />` and `<SignUpForm />`. Write the logic to POST credentials to the backend, wait for the token, and save it securely (e.g., in localStorage or secure cookies).
3. **Dual-Mode Ingestion UI:** Build a flexible, authenticated landing page to import code.
  - Provide a clean form to accept a manually pasted GitHub Repository URL.
  - Alongside this, integrate a "Connect GitHub" OAuth flow. Use the GitHub REST API to

- display a searchable native dropdown of their owned repositories.
  - Both methods must seamlessly trigger the POST /api/scans/github endpoint.
- 4. **Implement the Polling Mechanism:** Write the async logic to call GET /api/scans/{scan\_id} every 5 seconds. Update a dynamic progress bar as the status changes from PENDING → SCANNING → COMPLETED.
- 5. **Build the Results Dashboard (Kanban Upgraded):** Design the final view that maps through the verified vulnerabilities array. Upgrade this from a simple list to a rich data-grid or Kanban board. Allow users to filter vulnerabilities by Severity (Critical, High, Low) and sort by status (Open, Resolved, Ignored).
- 6. **Code Formatting & Interaction:** Implement a syntax highlighter component to cleanly display the Red Agent's Python script and the Blue Agent's patch. Add a button/toggle tied to a PATCH request so users can mark a bug as is\_resolved: true.
- 7. **Contextual AI Chat Assistant :** Build a slide-out Chat Panel attached to the vulnerability detail view. This allows the user to "Talk to AURIX" to ask follow-up questions about the specific bug (e.g., "Explain this SQL injection to me").
- 8. **One-Click GitHub Pull Requests ):** Add a "Remediate" button next to the AI's patch. Clicking this should automatically branch the user's repository, apply the AI's code patch, and open a Pull Request directly from your UI using the GitHub OAuth token.

## 4. Expected Output / Streamlining Flow

- A deployed (e.g., on Vercel or Netlify) web application that flawlessly handles user sessions, communicates with the Backend API, provides real-time loading feedback, and displays a beautiful, actionable security report.
- A user can log in, choose to quickly paste a public URL or sync their GitHub account, watch the dynamic progress bar, and then manage active threats in a Kanban board. They can chat with the AI and fix bugs with a single click via automated PRs.

## 5. Critical "Do Not Miss" Constraints

- **Syntax Highlighting is Mandatory:** Raw code strings look terrible in standard HTML and are unreadable. You *must* use a library (like react-syntax-highlighter) to render the poc\_script and patch\_code beautifully with proper indentation and colors.
- **Graceful Error Handling:** If the backend returns Status: FAILED, the UI must handle it gracefully (e.g., showing a red alert box: "Scan failed due to timeout") rather than crashing the page or loading infinitely.
- **Protected Routes:** Ensure that unauthenticated users are immediately redirected to your Login page if they try to access the dashboard.
- **Handle Incomplete Data Gracefully :** The AI might not always return a perfect patch\_code for every single vulnerability. Your UI must not crash if a field is null. Show a fallback UI (like "Manual Remediation Required").

## 6. Prerequisites & Skills Needed

- React.js / Next.js state management (Hooks, Context).

- REST API integration and asynchronous polling (e.g., using fetch, axios, or SWR).
- Tailwind CSS and modern UX/UI Design principles.
- Handling JSON Web Tokens (JWT) for user sessions.
- Integration with third-party APIs (GitHub REST API)

## 7. Reference Materials

### Docs provided by tech leads

- [aurix-HLD.mmd](#) or [png](#)
- [webDashboard-api-sequence-diagram.mmd](#) or [png](#)
- [database-er-diagram.mmd](#) or [png](#) (for mapping your UI data).

### External documentation

- [Next.js Documentation](#)
- [SWR or React Query \(for API polling\)](#)
- [React Syntax Highlighter](#)
- [GitHub REST API Documentation \(OAuth, Repositories, Pull Requests\)](#)

## 8. Component Build List & Utility

- **Authentication Screens (The Door):** Secures the app and identifies the user.
- **Dual-Mode Ingestion Form:** Triggers cloud-based scans via URL pasting or native GitHub repo selection.
- **Real-time Status UI:** Keeps the user informed and engaged during the 2-3 minute AI reasoning phase via polling.
- **Triage Kanban Board :** The primary workspace for filtering and managing active threats.
- **Vulnerability Detail View:** Displays the AI's reasoning, PoC scripts, and syntax-highlighted code.
- **Contextual AI Chatbot :** Acts as an on-demand security tutor for specific vulnerabilities.
- **Automated Remediation Module :** Handles the logic for the "One-Click PR" integration

## 9. CRITICAL DEPENDENCY (You are Blocked)

- **You are BLOCKED BY the Backend Engineer (Bhavya):** You cannot build your API calls, polling logic, or login flow without her endpoints. **Your very first task** must be to demand "Mock APIs" from Bhavya on Day 1. She needs to provide you with fake, hardcoded JSON endpoints for /login and /scans so you can build your React components without waiting for the real database and AI to be finished.